

Process Increments: An Agile Approach to Software Process Improvement

Amr Noaman Abdel-Hamid

Software Engineering Competence Center (SECC)
IT Industry Development Agency (ITIDA)
Cairo, Egypt
anoaman@itida.gov.eg

Mohamed Amr Abdel-Kader

Software Engineering Competence Center (SECC)
IT Industry Development Agency (ITIDA)
Cairo, Egypt
mamr@itida.gov.eg

Abstract—*Process increments* is an iterative and incremental approach to manage software process improvement projects. It partitions the scope into user-story-like increments, and manages the whole project using agile estimating and planning techniques. This approach has been applied to five enterprises and showed sustained improvement velocity, good project visibility, and on-going learning experiences by project teams¹.

I. INTRODUCTION

Software process improvement projects incorporate organization transition risks, especially when improvement projects involve paradigm shift and organization restructuring towards Agile and lean processes. *Process increments* is an iterative and incremental approach to manage improvement projects. The approach builds upon Agile values and principles and reuses some well known Agile techniques in managing transition risks.

This approach has been applied to five small enterprises (fewer than 30 developers) in Egypt, two of them are adopting Agile methodologies and the other three are structuring themselves towards hybrid processes. The experiment shows better grasping of software engineering concepts, sustained improvement velocity throughout the projects, high team morale, better project visibility, and on-going learning experiences.

In this study, we will provide an overview of *the process increments* concept as a process improvement methodology. Then, we will provide full details about the implementation context, and how *process increments* were applied. Finally, we will display the results of the study, and present some interesting observations and findings.

II. PROCESS INCREMENTS

A. Definition

A *process increment* is a process improvement chunk which can be implemented in a relatively small time (1-2 weeks) and still provide value for the organization. A

process increment is independent of any other process increment, although it may have prerequisite ones.

The concept of *process increment* in software process improvement (SPI) projects is almost identical to user stories in Agile development projects.

B. Process Increment Attributes

Process increments have the following attributes:

- Summary title: A description of the process increment
- Verification points (aka Conditions of Satisfaction): These are the criteria which when satisfied imply that the process increment is fulfilled
- Size estimate: Similar to a user story, the size of a *process increment* is estimated in points, using techniques like Planning Poker
- Process area: Every *process increment* is categorized under one of the following broad process areas: (1) Management: includes increments whose main theme is related to management, like estimation, tracking, etc. (2) Product Development: includes increments whose activities mainly focus on developing the product, including testing activities. (3) Environment & Infrastructure: includes increments related to preparing the infrastructure necessary for development

C. Areas, Dependencies, and Priorities

Process increments have dependencies and/or prerequisite relationships among them. Although partitioning have taken into consideration that increments remain standalone and provide value for the organization, there is still some logic to be applied while ordering and implementing these increments.

Fig. 1 describes a map of *process increments*, drawn as rounded rectangles. Each of them may have one or more prerequisite increments, drawn as incoming arrows into the increment. Synchronization bars are used to describe the possibility of parallel application of increments.

¹ Refer to [1] and [2] for definitions of Agile techniques mentioned throughout this paper, like user stories, agile estimating and planning, burn charts, backlogs, velocity, etc.

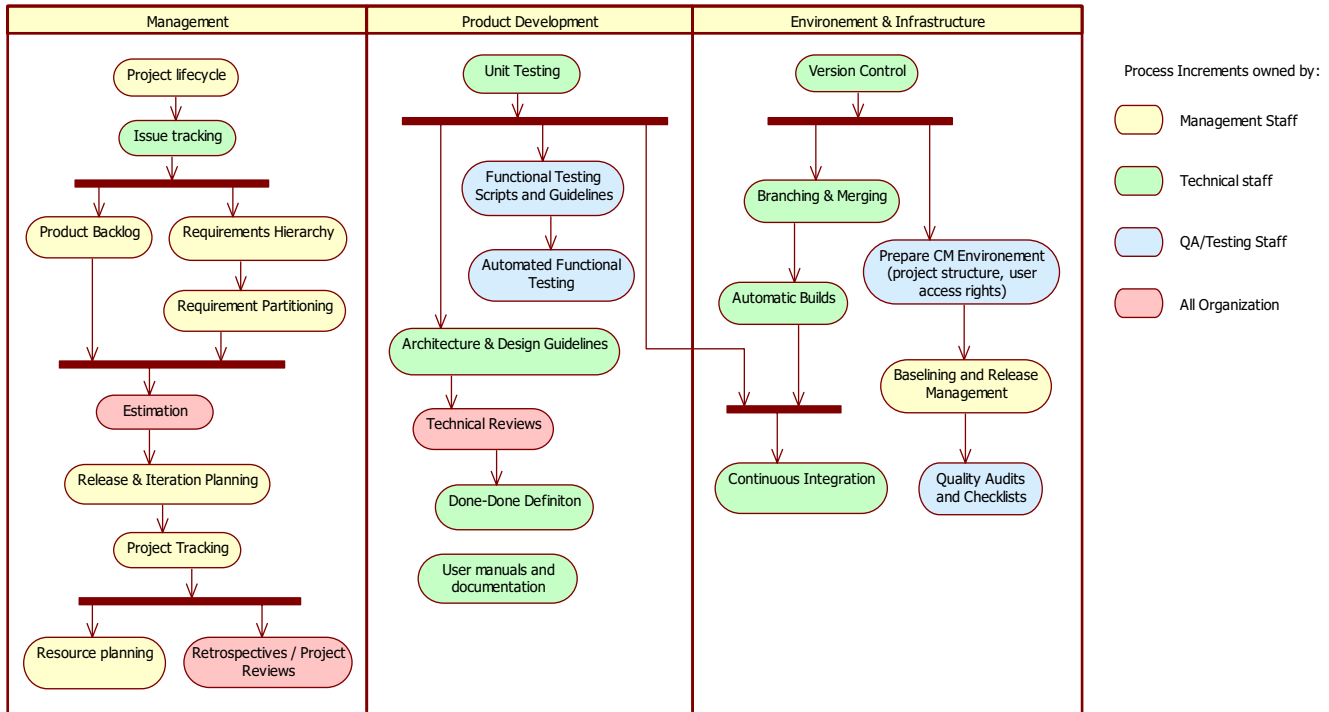


Figure 1. Process increments categories and dependencies

Also, the diagram shows in colors which role most probably will implement this increment:

1. Management: Senior managers, project managers, project leaders, etc.
2. Technical: Junior/senior engineering staff
3. QA/Testing: Testing engineers, quality assurance engineers (including persons responsible for process definition and process audits)
4. All the organization/team: Every team may own part of increment and apply it to his area of interest.

Typically, the assigned role owns the *process increment* and may do most of the investigations and studying. However, the whole team receives an overview about the increment and participates in implementing it. For example, if a project manager works on defining how release and iteration planning takes place, the whole team participates in the actual implementation of this process increment.

Process increments areas and dependencies are extremely useful in planning the SPI project. Project supervisors (usually the change agents in the organization and the consultant) prioritize the process increments based upon business priority (how urgent this increment is), increment prerequisites, and available resources.

Large *process increments* may be divided into smaller ones. For example, release & iteration planning was a big increment which had many activities and a long list of verification points. So, we divided it into release planning, iteration planning, and velocity tracking. Child increments are standalone and incur value even if the others are not implemented. However, it may be more appropriate to put

release planning as a prerequisite to both iteration planning and velocity tracking.

For a more comprehensive example, refer to the appendix for a listing of *process increments* covering the configuration management process area.

III. PROCESS INCREMENTS IMPLEMENTATION CONTEXT

The approach described above has been applied to five companies in a software process improvement program in Egypt. The following sections provide an overview about the motivation behind this program, and detailed implementation context.

A. Small and Medium Enterprises

Small and Medium Enterprises (SME) in Egypt have sizes with range from six persons to 40-50 persons. Some of them are growing and move across the boundaries from small to medium to large. Many of these enterprises have undergone process improvement attempts which were ad-hoc, casual, and/or impulsive.

We noticed that many of these attempts fail to continue, or fail to show return on investment [3] [4]². One of the reasons for these failures is applying international standards or methodologies with very little planning, no customization, and no vision of prerequisites and costs.

² In reference [3], the author noted that one fifth of the companies implementing CMMI-based process improvement initiatives abandoned the standard processes defined. Others complain from "Complexity of operations" and "Process overheads".

B. Software Process Improvement Program

To fill this gap, the Software Engineering Competence Center (SECC), a non-profit governmental agency, and one of the key drivers of Software Process Improvement in Egypt, developed the “Software Process Improvement for Small and Medium Enterprises” program (SPI for SME).

The program has three phases:

- Phase 1 – Training: The company team receives a series of training sessions on selected topics in software engineering (project management, requirements engineering, configuration management, software testing, technical reviews, and quality assurance)
- Phase 2 – Consultation & Coaching: The training sessions are followed by two-week improvement iterations for six months, which constitutes a 12-iteration process improvement project. These iterations are supervised and reviewed by a consultant (or better, a coach)
- Phase 3 – Evaluation: Companies participating in the program are formally evaluated against the program objectives and receive a final score and rank. Companies who do well in this program may receive further support from the IT Industry Development Agency (ITIDA) in Egypt

In this study, the companies’ development teams ranged from seven to 17 persons (including all roles). All of these companies are growing (some are growing very fast). Their business domains involve telecom, mobile applications, banking, enterprise applications, and government solutions. They adopt different technology stacks and programming languages like Java, .NET, Delphi, and C++. Table II summarizes some characteristics of these companies.

TABLE I. PROFILES FOR COMPANIES PARTICIPATING IN THE STUDY

Company	Team size	Business domain	Customer	Technology
1	7	Telecom	Telecom providers	.NET, mobile applications
2	15	ERP, banking solutions	Public & private sectors	Delphi
3	13	Telecom	Telecom providers	Java, C++
4	13	Enterprise applications	Government	.NET
5	17	Enterprise applications, ERP	In-house, and private sector	.NET, Java

C. Agile Techniques Employed

Many Agile techniques have been employed while implementing this program. The following is a listing of Agile practices used:

1) *Process Cards*: All process increments are printed on process cards (Fig. 2), with the size estimates of each process increment, and verification points printed on the back.

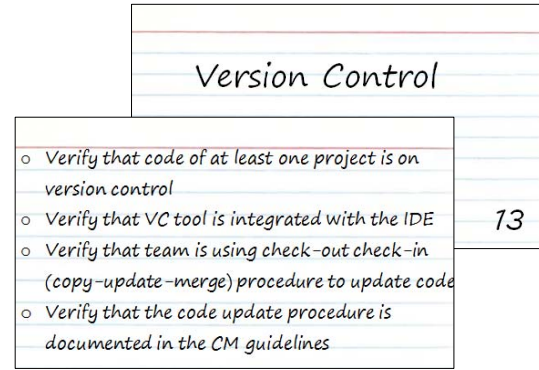


Figure 2. Process card for the ‘Version Control’ process increment

Process cards are used in the planning phase. All process increments are inspected and prioritized. Then, a high level implementation plan is built accordingly.

2) *Iterative Lifecycle*: Process improvement can introduce major organization changes, and these changes should be managed against risks which may occur. The iterative lifecycle is one of the excellent techniques to manage these risks. Typical improvement iteration is described in Fig. 3.

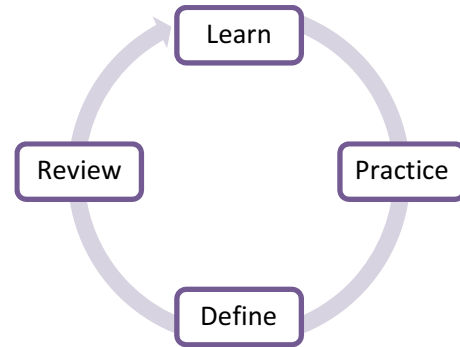


Figure 3. Typical two-week iteration in the improvement project

This is a two-week iteration which contains the following activities:

- **Learn** new concepts. These concepts might be new process increments, or further detailing of a process increment carried over from previous iterations. This learning process is time-boxed to one day.
- **Practice** this new concept and apply it to live projects. This is the major part of the iteration time. The team passes through all the pain of trying new tools, concepts, procedures, etc.
- **Define** what is practiced into guidelines or process documentation. The intent of this activity is to record what the team has learned and practiced to be used while rolling out changes throughout the rest of the organization. Also, most companies are suffering from high rate of turn-over. Developers or testers who participated in this learning experience

may leave the company shortly after the program ends. Finally, part of the program is introducing the quality audits into the organization. This is not very common in Agile environments. However, it is recommended when the culture of the working staff is loose and overly relaxed. In this case, high level process guidelines would be excellent criteria for a quality auditor to assess whether the team is implementing what they agreed to be good practices.

- **Review** what has been accomplished with the consultant, and evaluate whether the process increment is done-done or not. The definition of 'done' is presented below.

3) *Product Backlog*: In the SPI project, two backlogs are used:

- The project backlog, which contains all the process increments. This backlog keep expanding and shrinking according to the needs of the organization. This is why the total project scope in Table II is differs from one organization to another.
- The iteration backlog, which contains process increments and tasks to be completed during this iteration.

4) *Information Radiator*: Information radiators have proven to be a very effective technique for visualizing information and tracking progress in software projects; and therefore they were used in the process improvement project. Fig. 4 shows the Iteration backlog populated with process increments (yellow notes). Increments are divided into tasks (green notes, and may become red if they are late); and the improvement progress is shown by the burn-up chart on the right



Figure 4. Board showing process increments, tasks, and burn chart

5) *Burn-up Charts*: Project burn-up chart is used to track project progress (in terms of size points). This chart played a positive role in motivating the team, and increasing awareness and visibility of the SPI project.

6) *Done-Done Definition*: The done-done definition is the minimum conditions required so that a process increment is considered completed. The definition ensures the quality of implementing process increments. In the SPI project, we use this definition of 'done-done':

1. All verification points are fulfilled
2. The *process increment* is applied to at least one live project
3. The *process increment* is documented in a process guideline, or described in one of the organization process standard documentation

7) *Improvement Velocity*: Improvement velocity is used to measure the progress of the team improvement. It is calculated by summing the size points of increments which are done-done in the iteration.

IV. RESULTS

A. Velocity Readings

Table III displays a summary of company results. The first column is the total scope for every company. The second column is the total amount of done-done process increments out of this scope. The last column is the average improvement velocity.

TABLE II. VELOCITY READINGS

Company	Total Scope (points)	Total Completed (points)	Accomplished	Average Velocity (points/iteration)
1	251	182	72.5%	18.2
2	301	231	76.7%	23.1
3	283	278	98.2%	25.3
4	297	268	90.2%	24.4
5	230	222	96.5%	20.2

The first two companies who accomplished relatively lower percentages were stuck with other business priorities, or suffered from technical problems while setting up the configuration management environment.

B. Observations & Findings

1) Improvement Velocity

Improvement velocity was comparable across organizations, which indicates a successful method of improvement. The companies' circumstances, products, technologies, customers are different. They have gone through issues and obstacles which are also different. Refer to Table II for average velocities of the companies during the project duration.

Another reason why velocities were comparable is the parallel paths of implementation. At any point of time, there were active increments falling under the three major process areas (management, product development, and environment & support). If one path slowed down due to any impediments, the other two continues and the organization preserves a good improvement velocity, especially since the persons working on those process areas are usually different.

The graph in Fig. 5 shows average velocity (for all organizations) calculated for every iteration. As appears, the average velocity is sustained throughout the 12 iterations of the software improvement project, which indicates good accomplishment rate throughout the project.

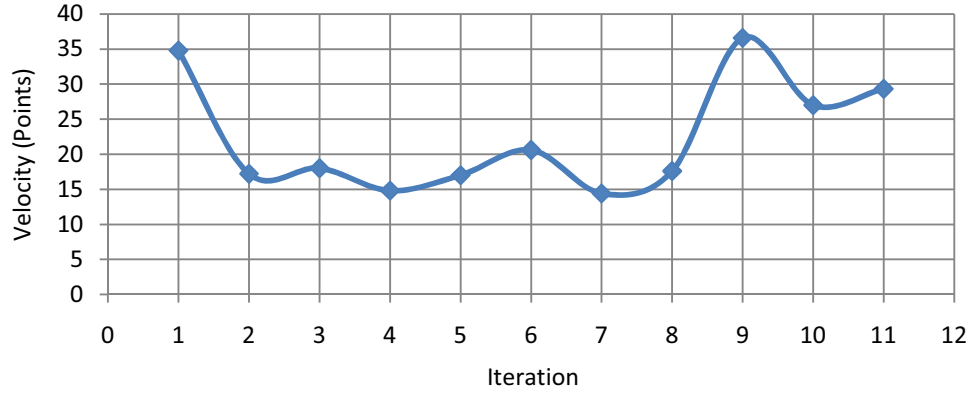


Figure 5. Average velocity (for all organizations) throughout the improvement project

2) Role-based improvement

Sometimes, managers think that the technical staff are not interested in process improvement. In other cases, managers argue that the technical staff are too busy to participate. In our experiment, we have marked every process increment by owner role who actually implemented most of it. Fig. 6 shows the contribution for every role.

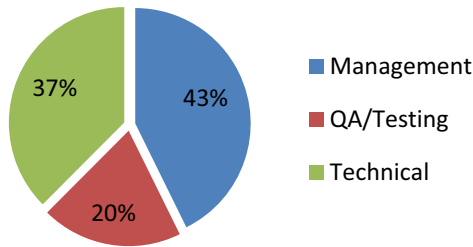


Figure 6. Contribution of role categories in the improvement project

The major finding here is that the contribution of the development team (technical and QA/Testing) is about 63% of the overall improvement. This may give an indication why improvement projects fail when run in the management silo only.

Generally speaking, there is no dominant role in the actual process increments implementation³. The contribution of the management staff (project managers/leaders) is more or less comparable to the contribution of the technical staff (Junior/senior developers, technical leaders, etc.). Also, although the testing/QA contribution is smaller, it still accounts for one fifth of the actual improvement.

3) Progress Visibility

Improvement progress was visible all over the organization. First, the whole organization staff knew exactly how well they were doing. Also, senior managers used to pass by the burn-up chart and discuss the trend of the

³ For some, the dominant role is a steering committee or an SEPG [5]. For others, it may be purely owned by PM's or senior managers. In contrary, in other cases, we heard a lot of comments indicating the development team (the technical people) are the ones who really does the "revolution" in software process improvement, etc.

improvement progress. They also referred to this chart for pinpointing problems and possible acceleration of improvement.

The burn chart was intentionally placed in a place visible to all project teams (in the development area, or may be in the corridor driving to the development area). Other teams not participating in process improvement effort were eager to embrace process increments or even participate as pilot projects.

4) Learning by example

Driving the project as an agile project has aided the project team to understand basic concepts of Software Engineering. For example, requirements hierarchy and partitioning, size estimation, iterative lifecycle, and iteration planning and tracking are all software engineering practices employed in the process improvement project. All companies (even those not adopting an agile process) have adopted some or all of the practices used in the process improvement project.

5) Team Morale

Team morale was very high. The fact that all the team (management, technical, support, QA & testing) are participating in the improvement effort was very effective.

V. CONCLUSION

Process increments is an excellent methodology to implement and manage process improvement projects. It is built over Agile concepts, reuses Agile techniques to manage the organization transition risks, and minimize the probability of failure.

The study conducted on a sample of five enterprises showed sustained improvement velocity, although the enterprises are of different backgrounds, technologies, and business areas; and faced different obstacles and issues.

Teams participated in the program indicated high team morale, senior management had better project visibility, and the whole organization have gone through an on-going learning experience.

Further experiments will concentrate on how to customize the process increments to match the specific nature of small enterprises in Egypt. Other experiments are being implemented to conduct CMMI-based process

improvement using *process increments*. In all ongoing and future experiments, we will track effort as well as size, and this will enable us to precisely calculate and study and benchmark the costs of improvement compared to the outcome.

ACKNOWLEDGEMENT

We would like to thank Amr Kamel, Department of Computer Science at Cairo University, Egypt, for his support and helpful review. We would also like to thank Ian Savage, McAfee, Inc. for his review and insight about agile adoption projects.

APPENDIX – SAMPLE PROCESS INCREMENTS

Table III lists increments fully covering the area of configuration management. It shows how large process areas are partitioned into process increments, and how verification points are written for every increment. It also lists the size of the increment in points, which were group estimated by three experts in the field using the planning poker technique.

In software process improvement projects, this technique was very effective in clearly identifying the scope. It also enabled tracking progress of the SPI project using value-based techniques like release burn charts.

TABLE III. CONFIGURATION MANAGEMENT PROCESS INCREMENTS

Title	Verification Points	Size Points
Issue Tracking	- Verify that a tool is selected for issue tracking - Verify that Requirements, Bugs, and CR's are tracked for at least one project - Verify that the issue tracking tool and the version control tool are integrated	13
Version Control	- Verify that code of at least one project is one version control - Verify that VC tool is integrated with the IDE - Verify that team is using check-out check-in (copy-update-merge) procedure to update code - Verify that the code update procedure is documented in the CM guidelines	13
CM Environment	- Verify that CM environment is created - Verify that Backup/Restore procedure is defined and experimented successfully - Verify that users are defined and access rights are enforced - Verify that a default project structure is defined in the CM guidelines	13

Release Management	- Verify that branching and merging guidelines exist and used for one live project - Verify that versioning scheme is defined and employed - Verify that a packaging, releasing, and post-release verification procedure is defined and employed	8
Baselining	- Verify that baselines are identified across the project lifecycle - Verify that the baselines contents (work products) are identified - Verify that a specific location for baselines is identified on the CM tool - Verify that team has no update right on the baselines directory - Verify that naming conventions for the baselines are documented and used	5
Traceability	- Verify that requirements are traceable to their design, code modules, and test cases - Verify that requirements are traceable to defects and CR's - Verify that traceability is bi-directional	5
Deployment Process	- Verify that a procedure exists for communicating packages to deployment engineers - Verify that deployment environment is defined, including software/hardware interfaces with deployed packages - Verify that a procedure exists for communicating deployment issues to the development team	8
Continuous Integration & Automatic Builds	- Verify that automatic build scripts are developed for at least one live project - Verify that automated tests run to validate generated builds - Verify that the development team are notified by the build status and generated issues (if any)	20

REFERENCES

- [1] Mike Cohn, *User Stories Applied*, Addison-Wesley, 2004.
- [2] Mike Cohn, *Agile Estimating and Planning*, Prentice Hall, 2005.
- [3] Alaa Hamouda and Mohamed El-wahsh, "Comparative Study of Software Engineering Processes in Egyptian CMMI Companies", The Journal of American Science, ISSN 1545-1003, Volume 6, Issue 11, Cumulated No. 32, November 1, 2010.
- [4] Ahmed Roshdy El Shalaby, "Capability Maturity Model (CMMI) Adoption by Egyptian Software Companies: Motivations and Barriers", unpublished MBA thesis, May, 2009.
- [5] Priscilla Fowler and Stan Rifkin, "Software Engineering Process Group Guide", <http://www.sei.cmu.edu/reports/90tr024.pdf>, pp.132